



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and
Technology

**CSC2210: OBJECT
ORIENTED PROGRAMMIN
G2 [Semester, Spring 2025-2026]
Section: [D]**

Project Report On
FoodX – Food Delivery Restaurant Management System

Supervised By
DR. MD. IFTEKHARUL MOBIN

Submitted By:

Name	ID
Rafsan Azad	25-60783-1
MD. Habibullah Tuhin	23-50617-1
Sabbir Hasan Tanim	23-50423-1
Shah MD. Zunaed	24-57769-2

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				Total =
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.				
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.				

Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.	
--------------	---	--

Dear Professor,

We respectfully submit this project report for the CSC2210: Object Oriented Programming 2 course. The project is a desktop-based Food Delivery Based Restaurant Management System developed using C# Windows Forms and SQL Server database integration.

The system brings together four major user roles—Super Admin, Restaurant Admin, Employee, and Customer—and provides modules for user management, restaurant management, category and food management, cart and ordering, payment handling, employee order processing, and reporting.

The report follows the structure of the supplied reference report while documenting the actual FoodDeliverySystem project materials provided with this submission.

Best Regards, Project

Team

Table Of Contents

Topics	Page No.
1. Title	4
2. Introduction	4
3. Case Study	4
4. ER Diagram	5
5. Normalization & Finalization	6
6. Database Schema	9
7. Database Query	10
8. Transition Table (Screenshots)	15
9. Features and Facilities	27
10. Conclusion	27

1. Title

FoodX – Food Delivery Restaurant Management System.

2. Introduction

Ordering food online has become part of everyday life, and the value a customer gets from the experience depends heavily on how well the restaurant, its staff, and the delivery process are coordinated behind the scenes. This project, FoodX, is a desktop-based Restaurant Management System built with C# Windows Forms and a SQL Server database. It brings together four kinds of users – a Super Admin who oversees the whole platform, Restaurant Admins who manage their own restaurant(s), Employees who process incoming orders, and Customers who browse, order and pay for food – into a single, role-based application.

The system was designed around the core Object-Oriented Programming 2 course outcome of building a real-life, GUI-based, database-integrated desktop application, and it demonstrates the full life cycle of an order: from a restaurant listing its menu, to a customer building a cart and checking out, to kitchen staff accepting and fulfilling the order, to the Super Admin reviewing platform-wide sales.

3. Case Study

FoodX connects three sides of the food delivery ecosystem – restaurant owners, restaurant staff, and customers – under the supervision of a platform Super Admin. Here is how the system works in practice:

Rafsan Azad is the Super Admin of FoodX. He can see every restaurant on the platform, every registered user, every order that has been placed, and the total revenue generated across the system. From his dashboard he can activate, suspend, or delete user accounts, manage restaurant records, and maintain the shared list of food categories (Burger, Pizza, Biryani, Beverage, Dessert, and so on).

Habibullah Tuhin is a Restaurant Admin who owns two restaurants, Tuhin Kitchen and Tuhin Snacks Corner. From his Restaurant Panel he updates his restaurant profile, adds and prices menu items, keeps an eye on low-stock items, hires employees, and reviews how many orders each of his restaurants has received and how much revenue they have earned.

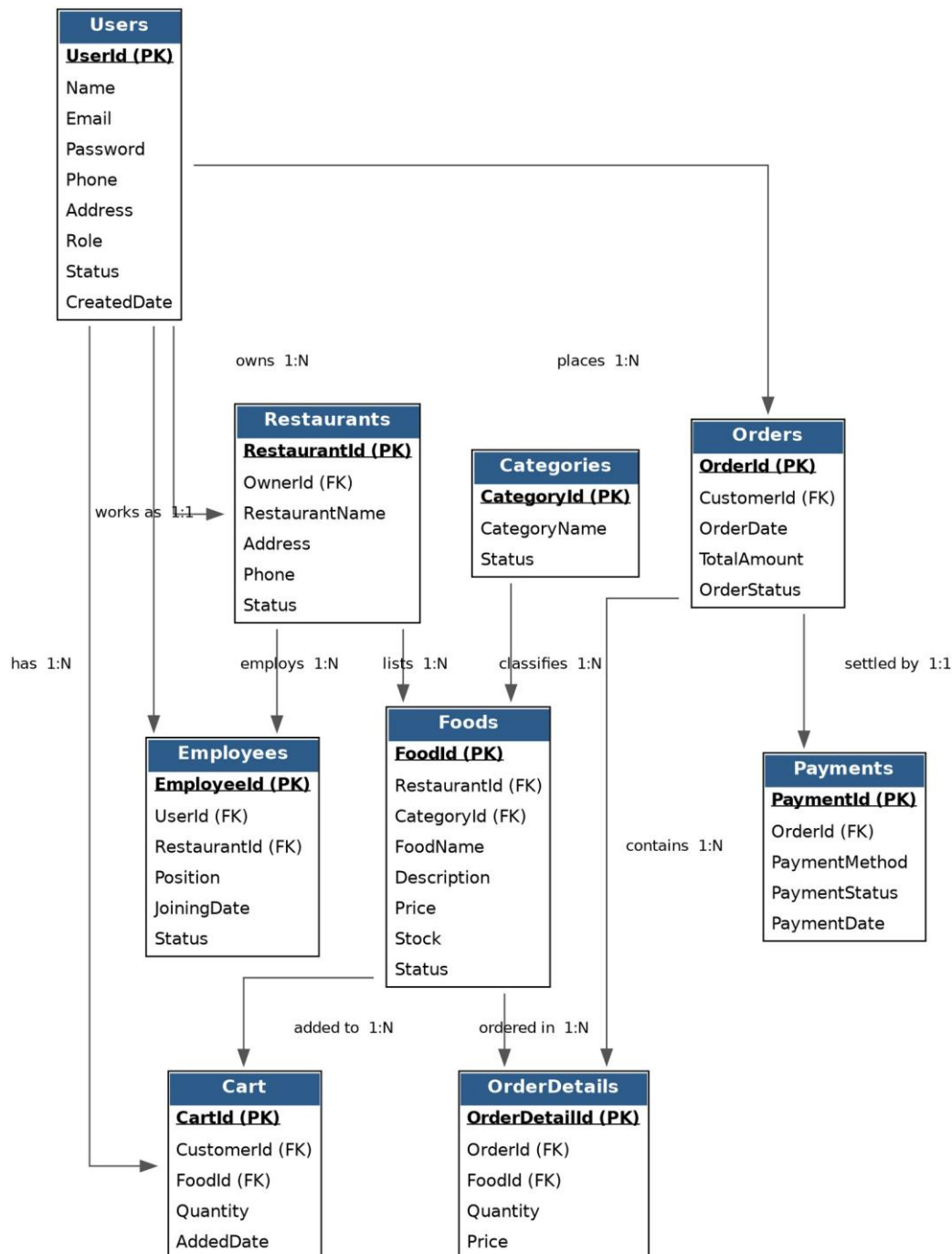
Sabbir Hasan and Shah MD. Zunaed work as employees under Tuhin's restaurants. When a customer places an order, it lands on their Employee dashboard as a Pending order. They review the order, change its status through the pipeline (Accepted → Preparing → Completed), print a kitchen ticket, and mark the payment as received once the order is delivered and paid for.

Ayesha Rahman, Nafis Iqbal, and Tanvir Ahmed are customers. Ayesha logs in, browses the available food items by category and price, adds items such as a Classic Beef Burger and a Cold Coffee to her cart, checks out by choosing a payment method, and receives an on-screen order confirmation and a printable invoice. She can later track her order history and review the status of every order she has placed.

Throughout this process, FoodX keeps every party in sync: restaurant owners see accurate stock and sales figures, employees only see the orders relevant to their own restaurant, and customers always know the current status and cost of what they have ordered. This coordinated flow of data between the Super Admin, restaurant owners, employees, and customers is the core problem FoodX was built to solve.

4. ER Diagram

The diagram below shows the conceptual entity–relationship model of FoodX, derived directly from the RestaurantDB schema used by the application. Primary keys are shown underlined and in bold; foreign keys are labelled (FK).



5. Normalization & Finalization

Each relationship in the ER diagram is normalized below, starting from the unnormalized (UNF) combination of attributes through to Third Normal Form (3NF).

Owns: Users → Restaurants (one-to-many).

UNF: UserId, Name, Email, Password, Phone, Address, Role, Status, RestaurantId, RestaurantName, R_Address, R_Phone, R_Status.

1NF: UserId, Name, Email, Password, Phone, Address, Role, Status, RestaurantId, RestaurantName, R_Address, R_Phone, R_Status.

2NF:

1. UserId, Name, Email, Password, Phone, Address, Role, Status.
2. RestaurantId, RestaurantName, R_Address, R_Phone, R_Status, UserId (FK) as

OwnerId. 3NF: Same as 2NF.

Employs: Users + Restaurants → Employees (one-to-many).

UNF: EmployeeId, UserId, Name, Email, RestaurantId, RestaurantName, Position, JoiningDate, Status.

1NF: EmployeeId, UserId, Name, Email, RestaurantId, RestaurantName, Position, JoiningDate, Status.

2NF:

1. UserId, Name, Email (already stored in Users).
2. RestaurantId, RestaurantName (already stored in Restaurants).
3. EmployeeId, Position, JoiningDate, Status, UserId (FK), RestaurantId

(FK). 3NF: Same as 2NF.

Lists / Classifies: Restaurants + Categories → Foods (one-to-many).

UNF: FoodId, RestaurantId, RestaurantName, CategoryId, CategoryName, FoodName, Description, Price, Stock, Status.

1NF: FoodId, RestaurantId, RestaurantName, CategoryId, CategoryName, FoodName, Description, Price, Stock, Status.

2NF:

1. RestaurantId, RestaurantName (already stored in Restaurants).
2. CategoryId, CategoryName (already stored in Categories).
3. FoodId, FoodName, Description, Price, Stock, Status, RestaurantId (FK), CategoryId

(FK). 3NF: Same as 2NF.

Adds to Cart: Users + Foods → Cart (one-to-many).

UNF: CartId, CustomerId, CustomerName, FoodId, FoodName, Price, Quantity, AddedDate.

1NF: CartId, CustomerId, CustomerName, FoodId, FoodName, Price, Quantity, AddedDate.

2NF:

1. CustomerId, CustomerName (already stored in Users).
2. FoodId, FoodName, Price (already stored in Foods).
3. CartId, Quantity, AddedDate, CustomerId (FK), FoodId (FK). 3NF: Same as 2NF.

Places: Users → Orders (one-to-many).

UNF: OrderId, CustomerId, CustomerName, OrderDate, TotalAmount, OrderStatus.

1NF: OrderId, CustomerId, CustomerName, OrderDate, TotalAmount, OrderStatus.

2NF:

1. CustomerId, CustomerName (already stored in Users).
2. OrderId, OrderDate, TotalAmount, OrderStatus, CustomerId (FK). 3NF: Same as 2NF.

Contains: Orders + Foods → OrderDetails (one-to-many).

UNF: OrderDetailId, OrderId, OrderDate, FoodId, FoodName, Quantity, Price.

1NF: OrderDetailId, OrderId, OrderDate, FoodId, FoodName, Quantity, Price.

2NF:

1. OrderId, OrderDate (already stored in Orders).
2. FoodId, FoodName (already stored in Foods).
3. OrderDetailId, Quantity, Price, OrderId (FK), FoodId (FK). 3NF: Same as 2NF.

Settled by: Orders → Payments (one-to-one).

UNF: PaymentId, OrderId, OrderDate, TotalAmount, PaymentMethod, PaymentStatus, PaymentDate.

1NF: PaymentId, OrderId, OrderDate, TotalAmount, PaymentMethod, PaymentStatus, PaymentDate.

2NF:

1. OrderId, OrderDate, TotalAmount (already stored in Orders).
 2. PaymentId, PaymentMethod, PaymentStatus, PaymentDate, OrderId (FK).
- 3NF: Same as 2NF.

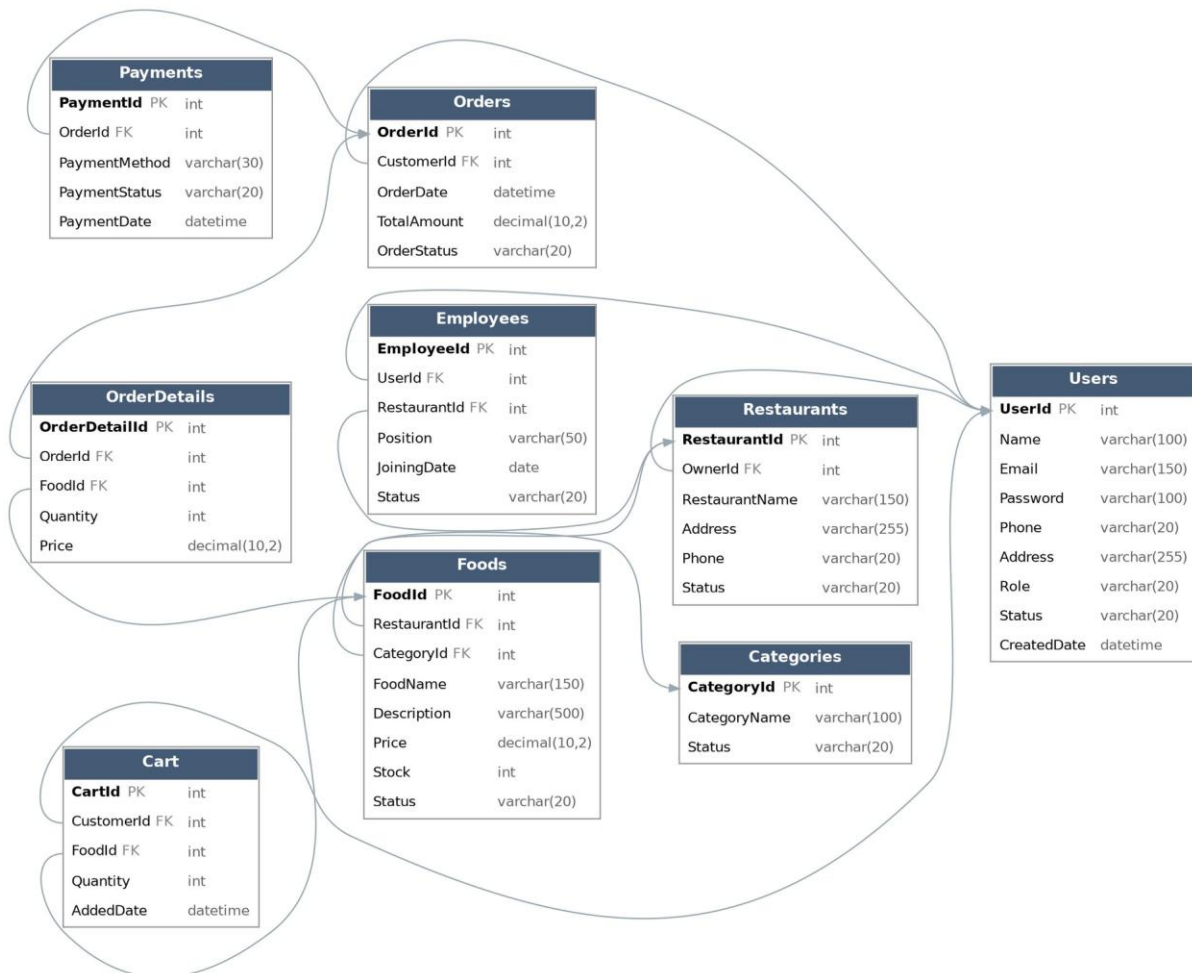
Finalization – Final Tables (3NF)

#	Table	Attributes
1	Users	UserId (PK), Name, Email, Password, Phone, Address, Role, Status, CreatedDate
2	Restaurants	RestaurantId (PK), OwnerId (FK), RestaurantName, Address, Phone, Status
#	Table	Attributes

3	Employees	EmployeeId (PK), UserId (FK), RestaurantId (FK), Position, JoiningDate, Status
4	Categories	CategoryId (PK), CategoryName, Status
5	Foods	FoodId (PK), RestaurantId (FK), CategoryId (FK), FoodName, Description, Price, Stock, Status
6	Cart	CartId (PK), CustomerId (FK), FoodId (FK), Quantity, AddedDate
7	Orders	OrderId (PK), CustomerId (FK), OrderDate, TotalAmount, OrderStatus
8	OrderDetails	OrderDetailId (PK), OrderId (FK), FoodId (FK), Quantity, Price
9	Payments	PaymentId (PK), OrderId (FK), PaymentMethod, PaymentStatus, PaymentDate

6. Database Schema

The physical database schema for RestaurantDB, generated from the final table design, is shown below. Lines indicate foreign-key relationships between tables.



7. Database Query

Database & Table Creation

Database

```
CREATE DATABASE RestaurantDB;  
USE RestaurantDB;
```

Users Table

```
CREATE TABLE Users  
(  
    UserId          INT IDENTITY(1,1) PRIMARY KEY,  
    Name            VARCHAR(100)      NOT NULL,  
    Email           VARCHAR(150)      NOT NULL UNIQUE,  
    Password        VARCHAR(100)      NOT NULL,  
    Phone           VARCHAR(20)        NULL,  
    Address          VARCHAR(255)      NULL,  
    Role            VARCHAR(20)        NOT NULL  
        CHECK (Role IN ('SuperAdmin','RestaurantAdmin','Employee','Customer')),  
    Status          VARCHAR(20)        NOT NULL  
        CHECK (Status IN ('Active','Inactive','Pending')),  
    CreatedDate     DATETIME           NOT NULL DEFAULT GETDATE()  
);
```

Restaurants Table

```
CREATE TABLE Restaurants  
(  
    RestaurantId     INT IDENTITY(1,1) PRIMARY KEY,  
    OwnerId          INT NOT NULL REFERENCES Users(UserId),  
    RestaurantName    VARCHAR(150) NOT NULL,  
    Address           VARCHAR(255) NULL,  
    Phone            VARCHAR(20)  NULL,  
    Status           VARCHAR(20)  NOT NULL  
        CHECK (Status IN ('Active','Inactive','Pending'))  
);
```

Employees Table

```
CREATE TABLE Employees  
(  
    EmployeeId       INT IDENTITY(1,1) PRIMARY KEY,  
    UserId            INT NOT NULL UNIQUE REFERENCES Users(UserId),  
    RestaurantId      INT NOT NULL REFERENCES Restaurants(RestaurantId),  
    Position          VARCHAR(50) NULL,  
    JoiningDate       DATE NOT NULL,  
    Status            VARCHAR(20) NOT NULL CHECK (Status IN ('Active','Inactive'))  
);
```

Categories Table

```
CREATE TABLE Categories
```

```
(
    CategoryId      INT IDENTITY(1,1) PRIMARY KEY,
    CategoryName    VARCHAR(100) NOT NULL UNIQUE,
    Status          VARCHAR(20)  NOT NULL CHECK (Status IN ('Active','Inactive'))
);
```

Foods Table

```
CREATE TABLE Foods
(
    FoodId          INT IDENTITY(1,1) PRIMARY KEY,
    RestaurantId    INT NOT NULL REFERENCES Restaurants(RestaurantId),
    CategoryId      INT NOT NULL REFERENCES Categories(CategoryId),
    FoodName        VARCHAR(150) NOT NULL,
    Description      VARCHAR(500) NULL,
    Price           DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
    Stock           INT NOT NULL CHECK (Stock >= 0),
    Status          VARCHAR(20) NOT NULL CHECK (Status IN ('Available','Unavailable'))
);
```

Cart Table

```
CREATE TABLE Cart
(
    CartId          INT IDENTITY(1,1) PRIMARY KEY,
    CustomerId      INT NOT NULL REFERENCES Users(UserId),
    FoodId          INT NOT NULL REFERENCES Foods(FoodId),
    Quantity        INT NOT NULL CHECK (Quantity > 0),
    AddedDate       DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT UQ_Cart_Cust_Food UNIQUE (CustomerId, FoodId)
);
```

Orders Table

```
CREATE TABLE Orders
(
    OrderId         INT IDENTITY(1,1) PRIMARY KEY,
    CustomerId      INT NOT NULL REFERENCES Users(UserId),
    OrderDate       DATETIME NOT NULL DEFAULT GETDATE(),
    TotalAmount     DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),
    OrderStatus     VARCHAR(20) NOT NULL
                    CHECK (OrderStatus IN ('Pending','Accepted','Preparing','Completed','Cancelled'))
);
```

OrderDetails Table

```
CREATE TABLE OrderDetails
(
    OrderDetailId   INT IDENTITY(1,1) PRIMARY KEY,
    OrderId         INT NOT NULL REFERENCES Orders(OrderId),
    FoodId          INT NOT NULL REFERENCES Foods(FoodId),
    Quantity        INT NOT NULL CHECK (Quantity > 0),
    Price           DECIMAL(10,2) NOT NULL CHECK (Price >= 0)
);
```

Payments Table

```
CREATE TABLE Payments
(
    PaymentId      INT IDENTITY(1,1) PRIMARY KEY,
    OrderId        INT NOT NULL REFERENCES Orders(OrderId),
    PaymentMethod  VARCHAR(30) NOT NULL
        CHECK (PaymentMethod IN ('Cash on Delivery','Card','Mobile Banking')),
    PaymentStatus  VARCHAR(20) NOT NULL CHECK (PaymentStatus IN
('Pending','Paid','Failed')),
    PaymentDate    DATETIME NOT NULL DEFAULT GETDATE()
);
```

Application Queries

7.1 Search Food (by name)

```
SELECT f.FoodId, f.FoodName, f.Price, f.Stock, r.RestaurantName, c.CategoryName
FROM Foods f
INNER JOIN Restaurants r ON f.RestaurantId = r.RestaurantId
INNER JOIN Categories c ON f.CategoryId = c.CategoryId
WHERE f.FoodName LIKE '%Burger%';
```

7.2 Filter Food (category + price + availability)

```
SELECT f.FoodId, f.FoodName, f.Price, f.Stock, r.RestaurantName, c.CategoryName
FROM Foods f
INNER JOIN Restaurants r ON f.RestaurantId = r.RestaurantId
INNER JOIN Categories c ON f.CategoryId = c.CategoryId
WHERE c.CategoryId = 1 AND f.Price BETWEEN 0 AND 500
      AND f.Stock > 0 AND f.Status = 'Available' AND r.Status = 'Active';
```

7.3 Join Query – Invoice Generation

```
SELECT od.OrderDetailId, f.FoodName, r.RestaurantName, od.Quantity, od.Price,
      (od.Quantity * od.Price) AS Subtotal
FROM OrderDetails od
INNER JOIN Foods f ON od.FoodId = f.FoodId
INNER JOIN Restaurants r ON f.RestaurantId = r.RestaurantId
WHERE od.OrderId = 1;
```

7.4 Group By Query – Super Admin Sales Report

```
SELECT r.RestaurantId, r.RestaurantName,
      COUNT(DISTINCT o.OrderId) AS TotalOrders,
      SUM(od.Quantity * od.Price) AS TotalSales
FROM Restaurants r
INNER JOIN Foods f ON r.RestaurantId = f.RestaurantId
INNER JOIN OrderDetails od ON f.FoodId = od.FoodId
INNER JOIN Orders o ON od.OrderId = o.OrderId
WHERE o.OrderStatus <> 'Cancelled'
GROUP BY r.RestaurantId, r.RestaurantName
ORDER BY TotalSales DESC;
```

7.5 Best Selling Food

```
SELECT f.FoodName, SUM(od.Quantity) AS UnitsSold,
      SUM(od.Quantity * od.Price) AS Revenue
FROM OrderDetails od
INNER JOIN Foods f ON od.FoodId = f.FoodId
GROUP BY f.FoodName
HAVING SUM(od.Quantity) >= 1
ORDER BY UnitsSold DESC;
```

7.6 Low Stock Report

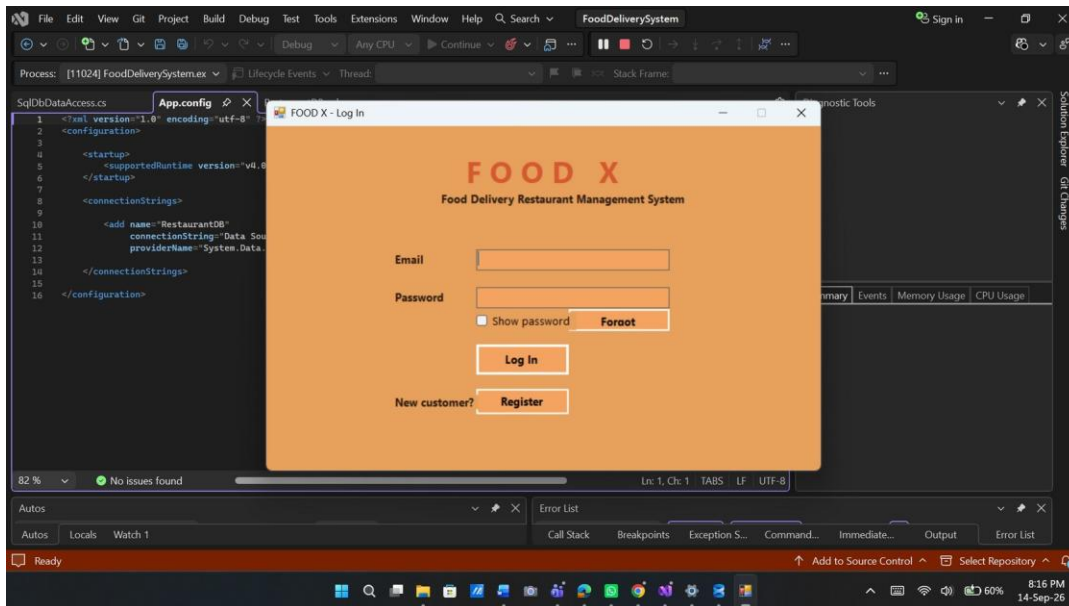
```
SELECT FoodId, FoodName, Stock  
FROM Foods  
WHERE RestaurantId = 1 AND Stock <= 10;
```

8. Transition Table (Screenshots)

The screenshots below walk through the running FoodX application – from login, through each role’s dashboard, to the customer ordering flow and the underlying database.

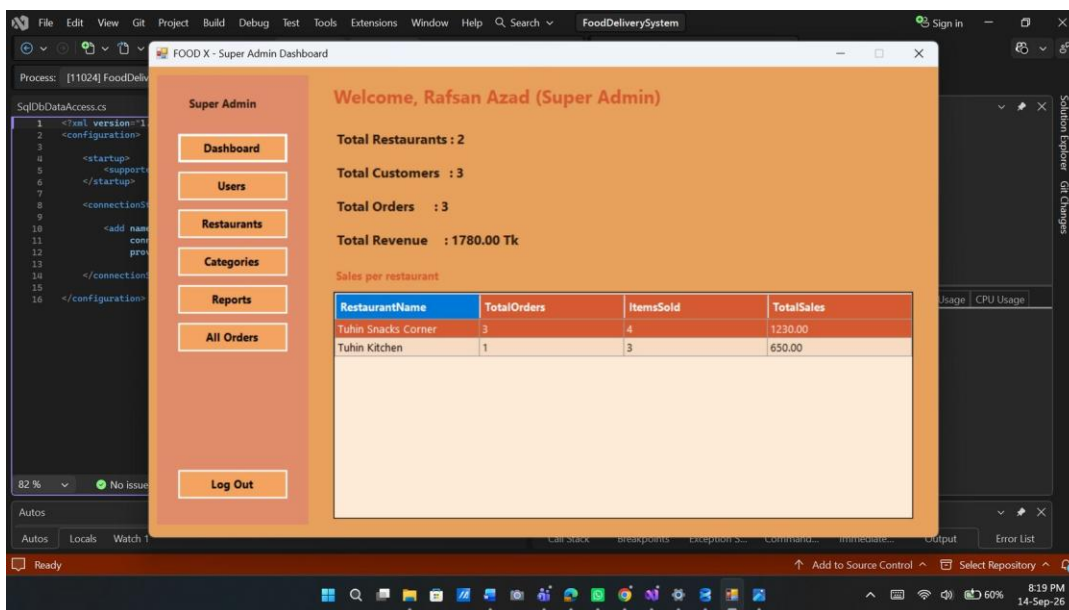
a) Login

Login screen – shared by Super Admin, Restaurant Admin, Employee and Customer.

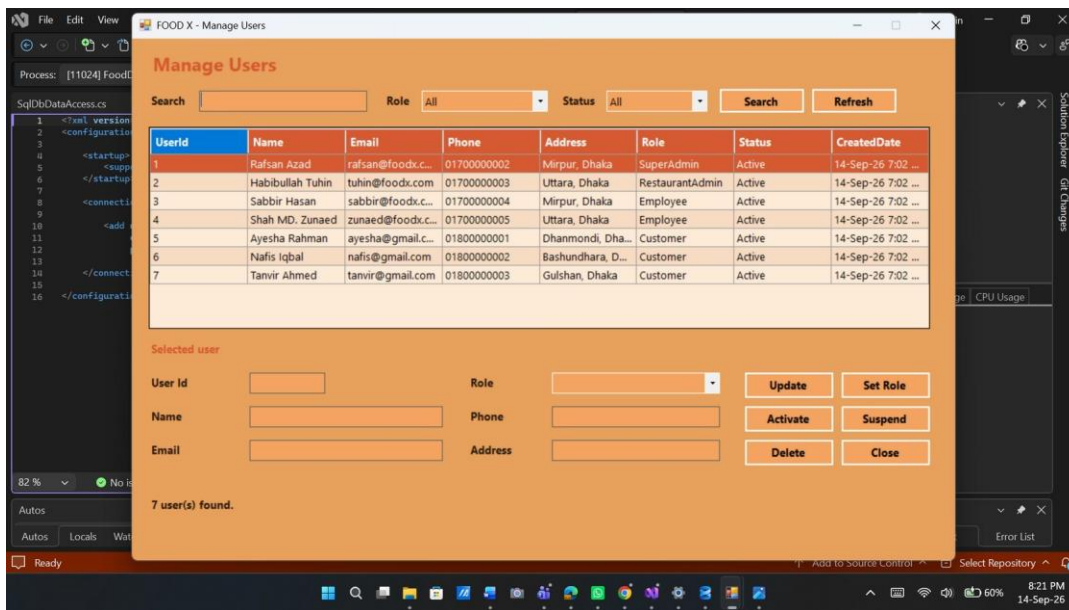


b) Super Admin Flow

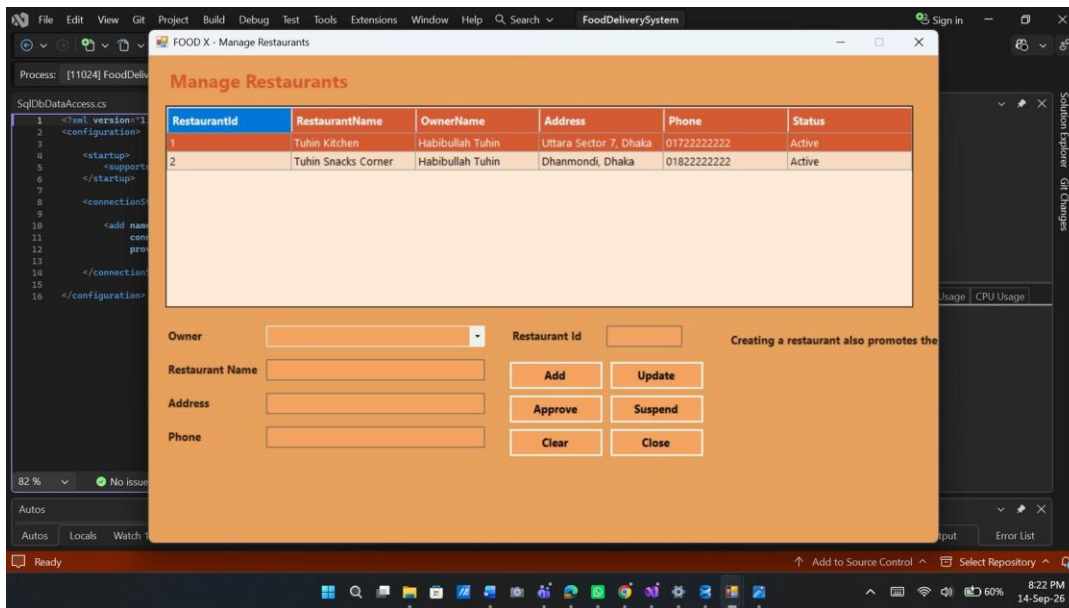
i) Super Admin Dashboard – platform totals and sales per restaurant.



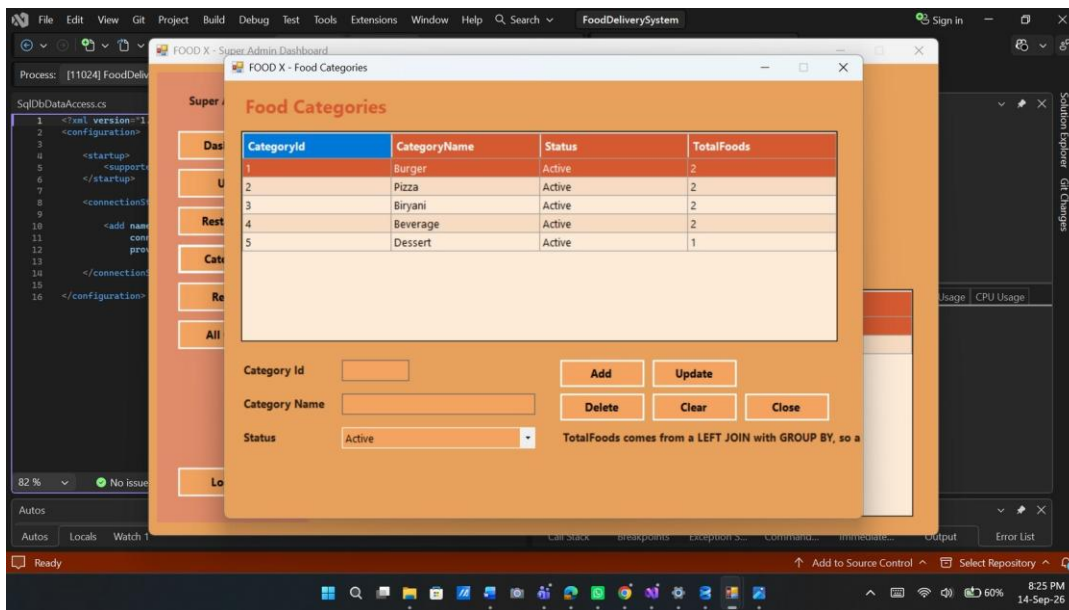
ii) User Management – search, activate/suspend, change role, or delete any user.



iii) Restaurant Management – view and manage every restaurant on the platform.

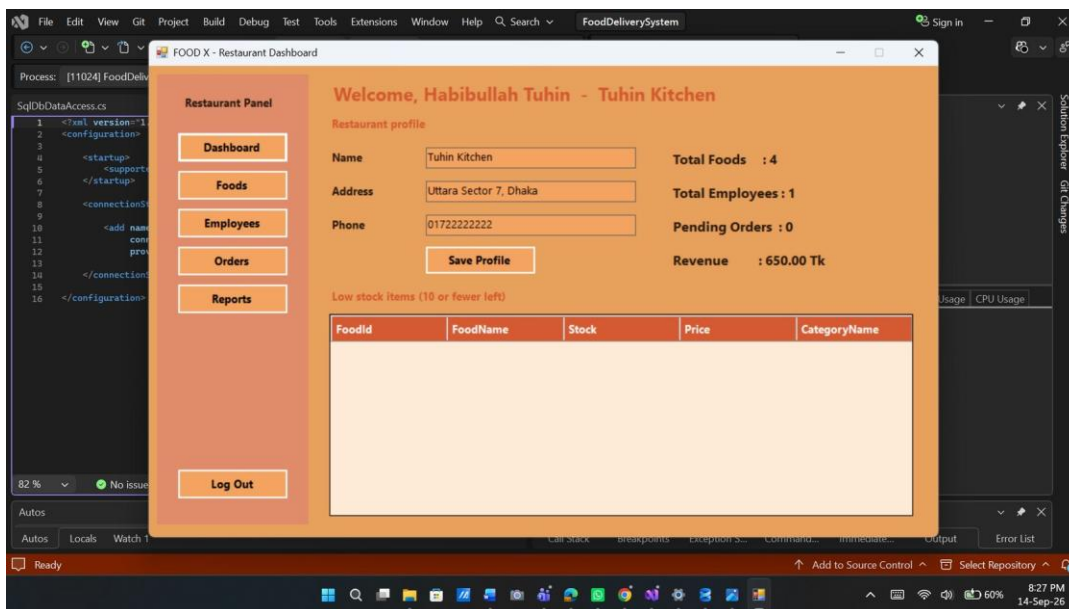


iv) Category Management – maintain the shared food category list.

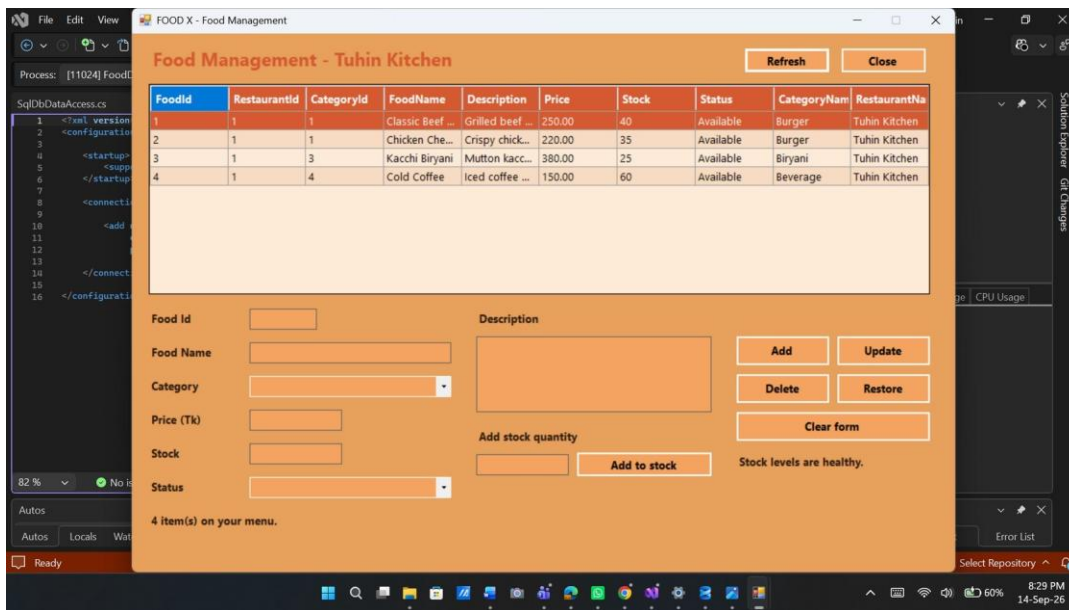


c) Restaurant Admin Flow

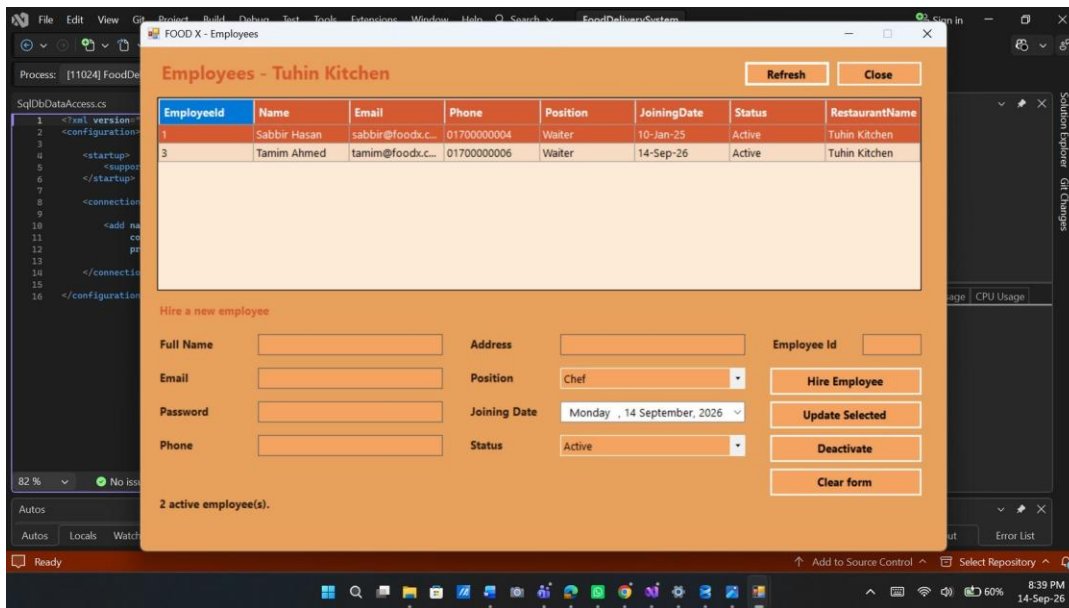
i) Restaurant Admin Dashboard – profile, revenue, and low-stock alerts.



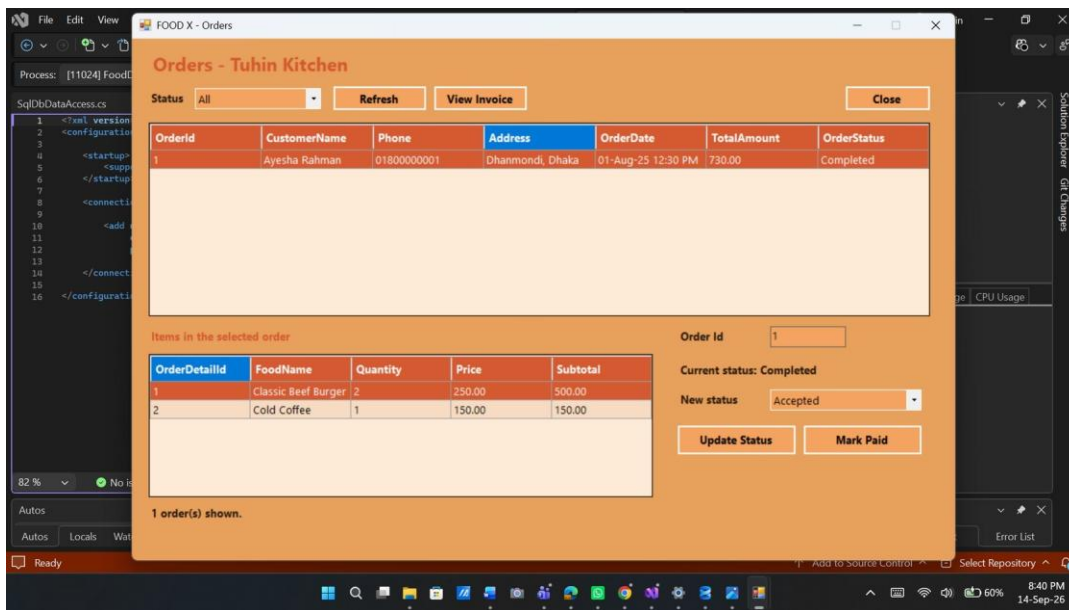
ii) Food Management – add, edit and price menu items.



iii) Employee Management – add employees to the restaurant.

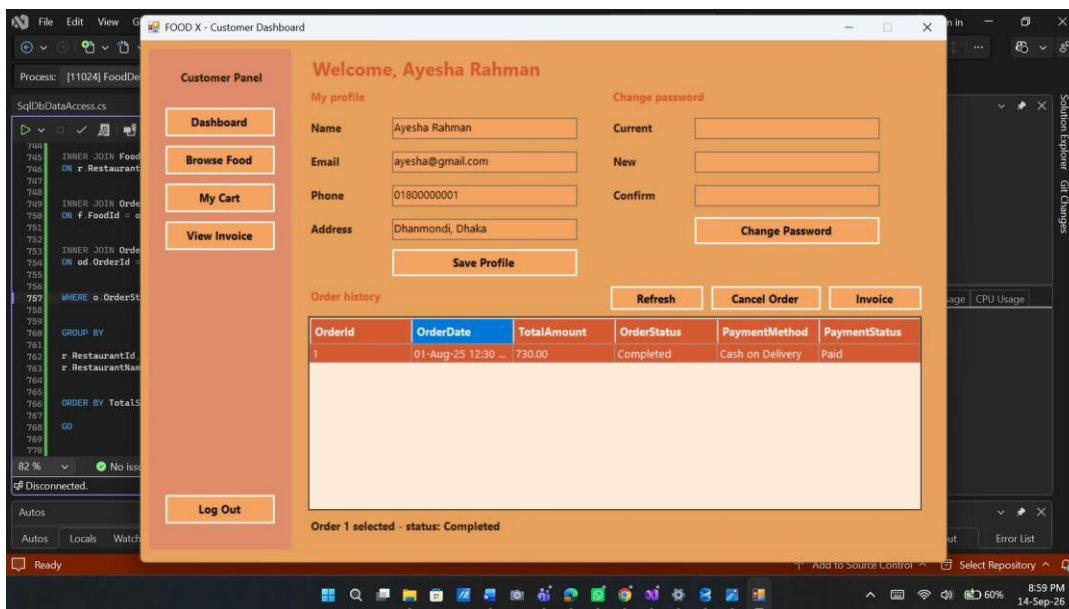


iv) Order Management – monitor every order placed against the restaurant.

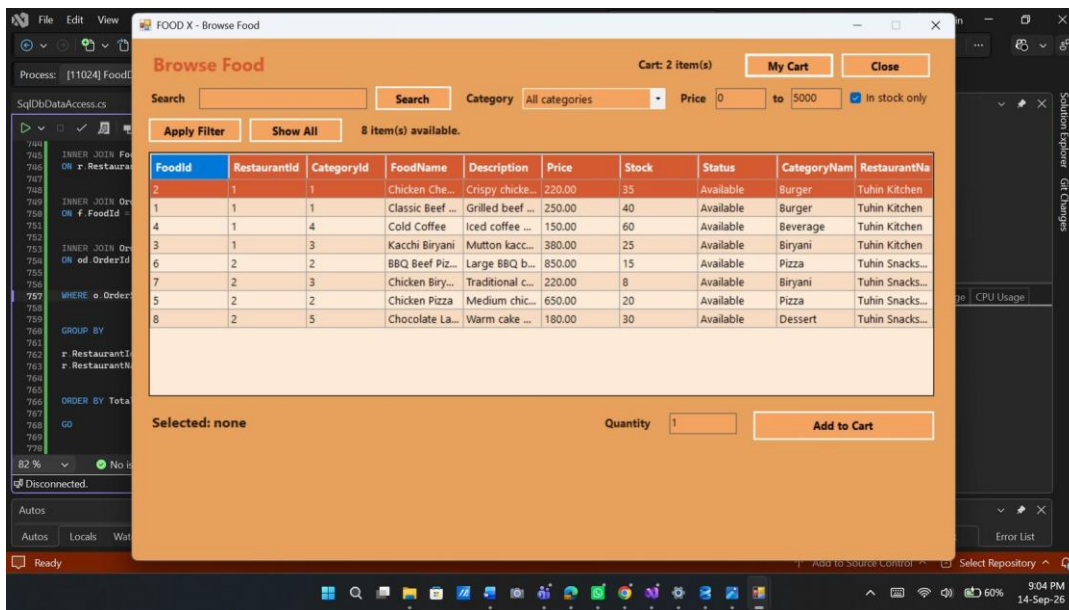


d) Customer Flow

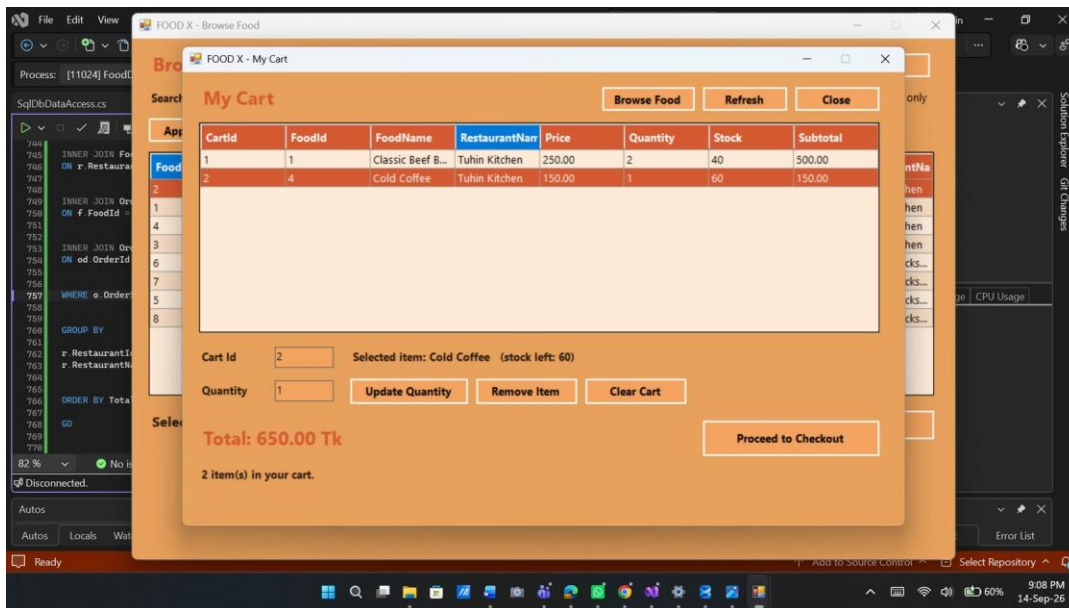
i) Customer Dashboard – profile, password change and order history.



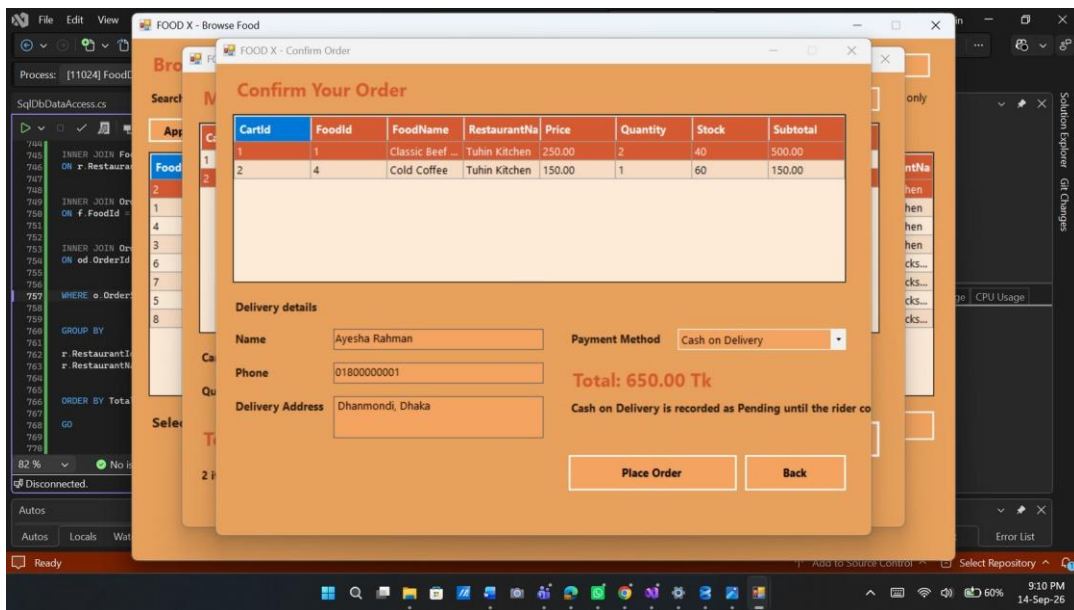
ii) Browse Food – search and filter menu items across restaurants.



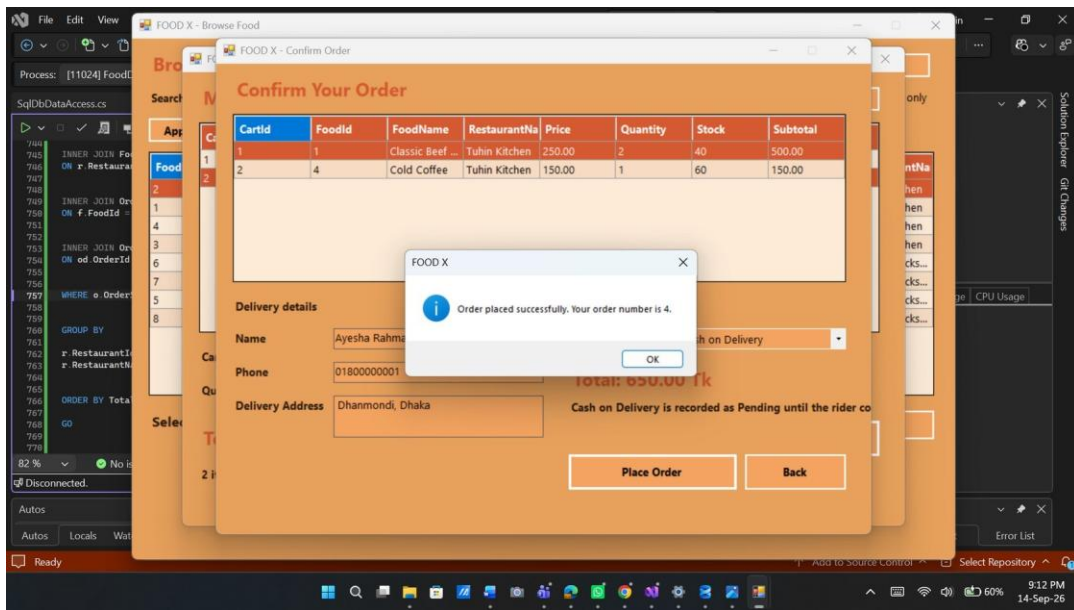
iii) My Cart – review items before checkout.



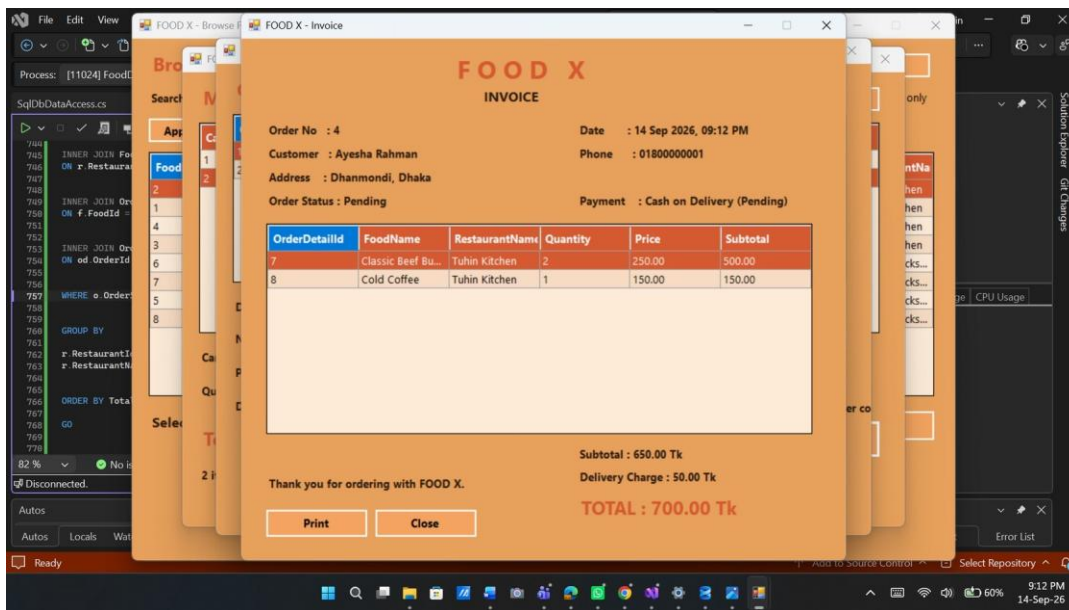
iv) Checkout – confirm delivery details and payment method.



v) Order Confirmation.

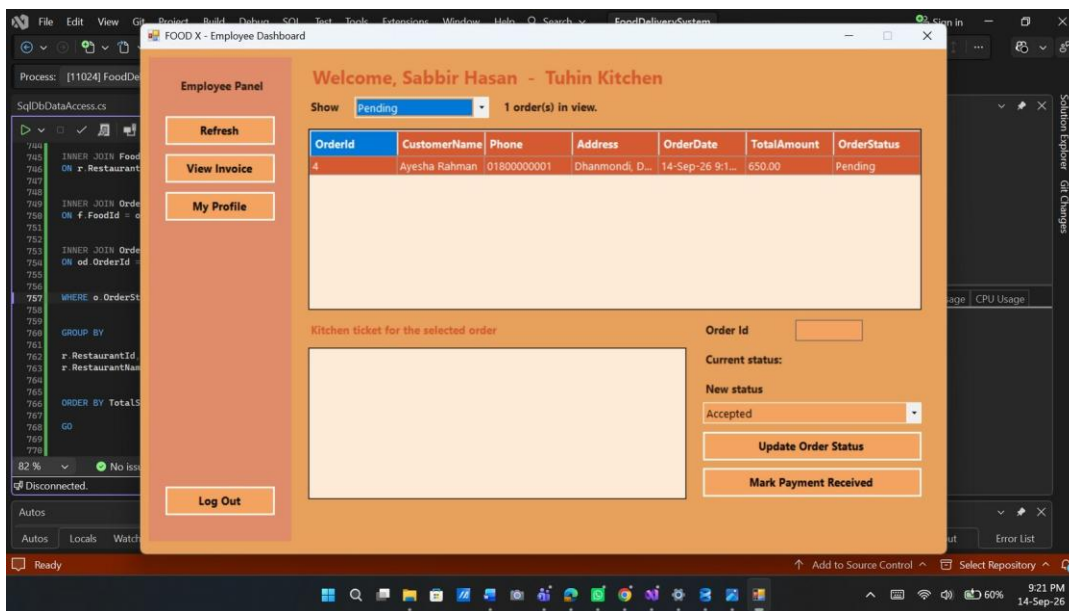


vi) Invoice – printable order summary.

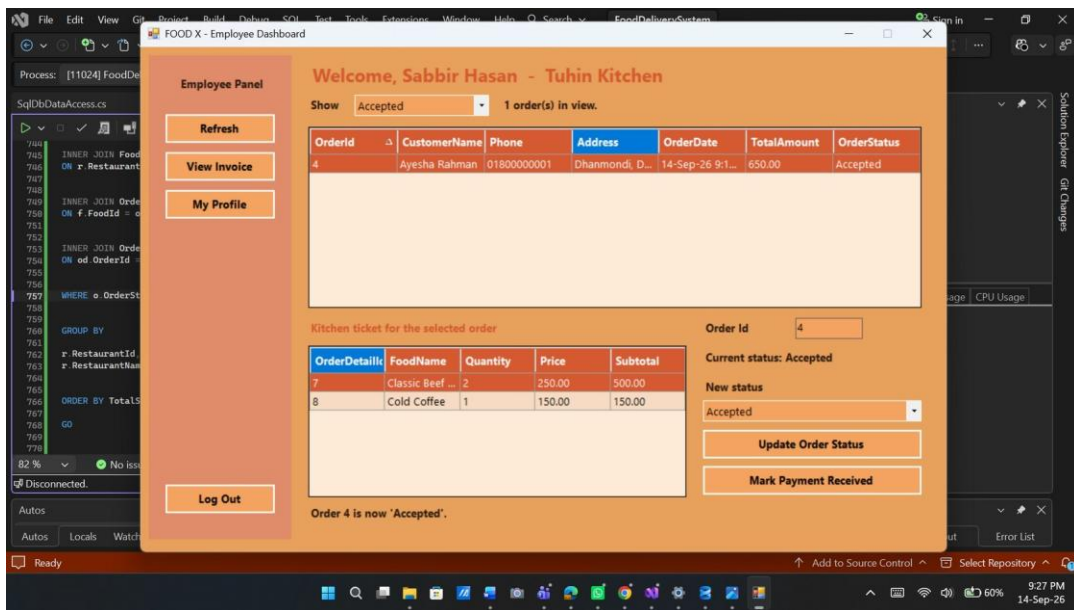


e) Employee Flow

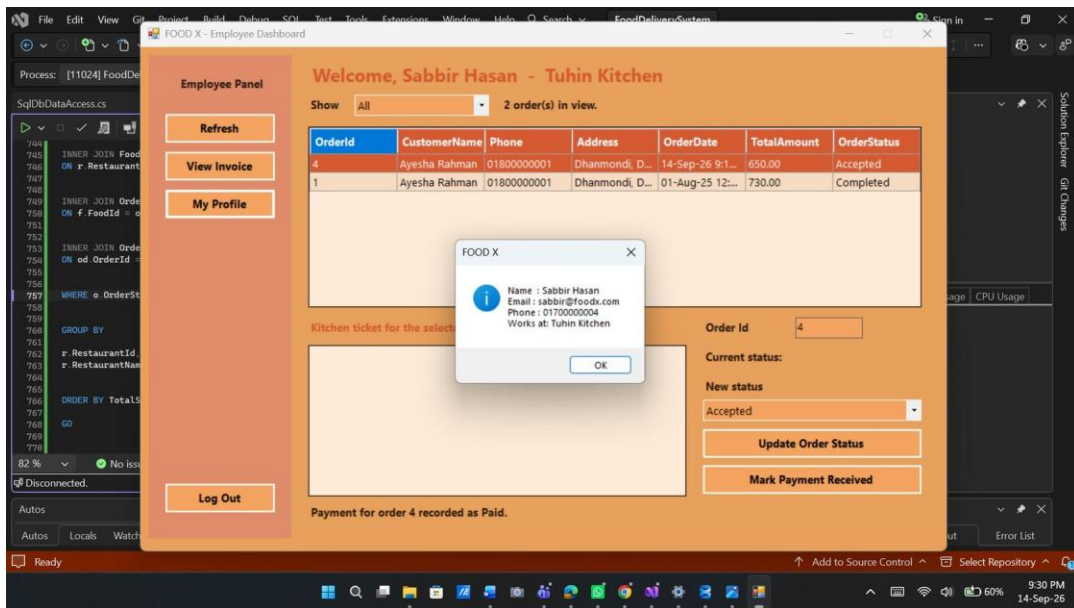
i) Employee Dashboard – pending orders for the restaurant.



ii) Accepted Order Details – kitchen ticket and status update.

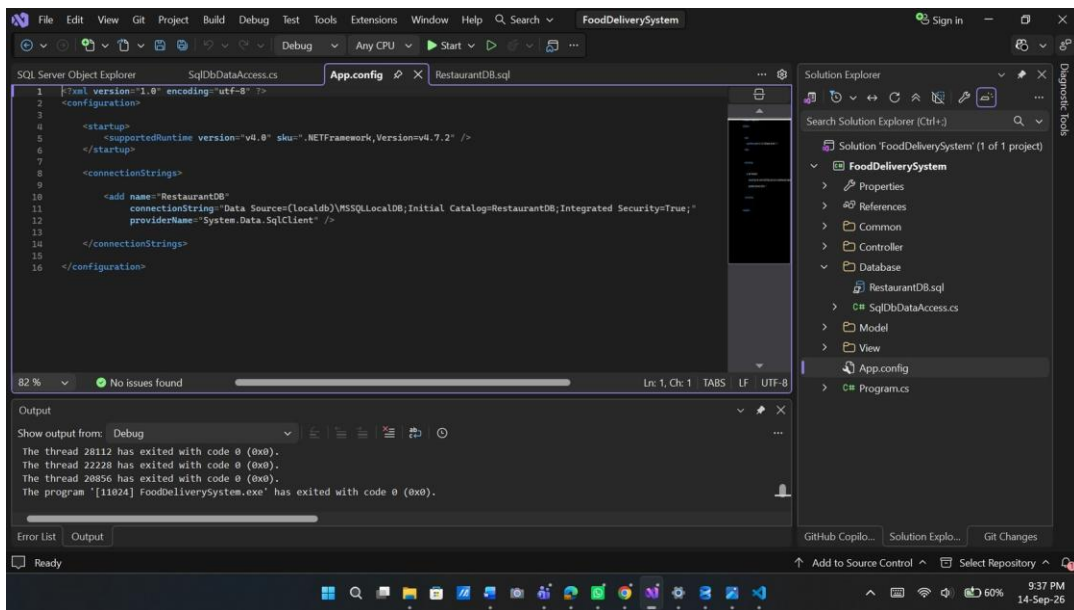


iii) Payment Received & Employee Profile.

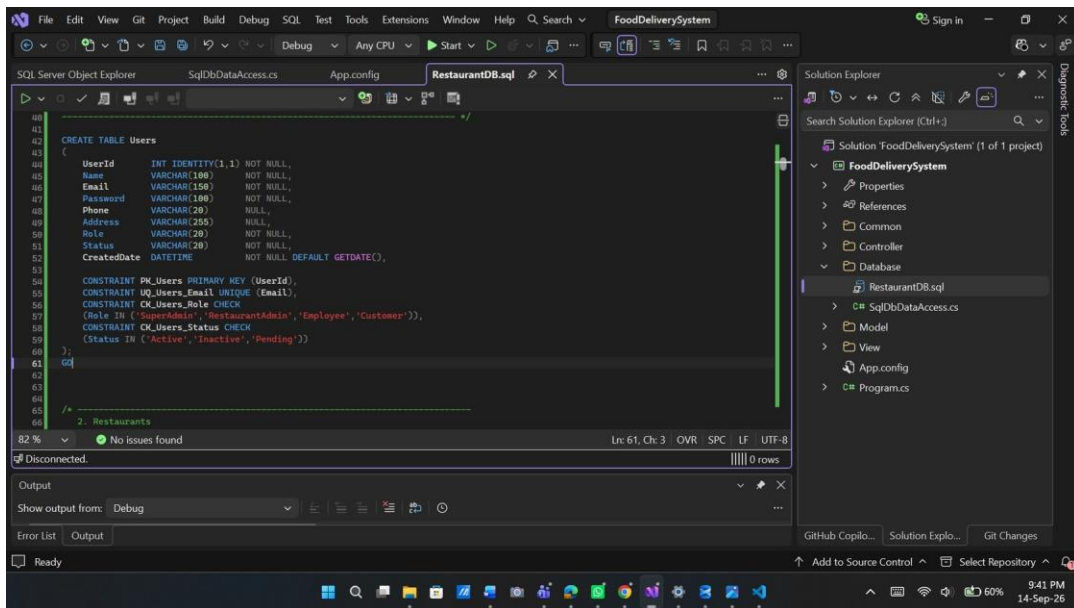


f) Database & Configuration

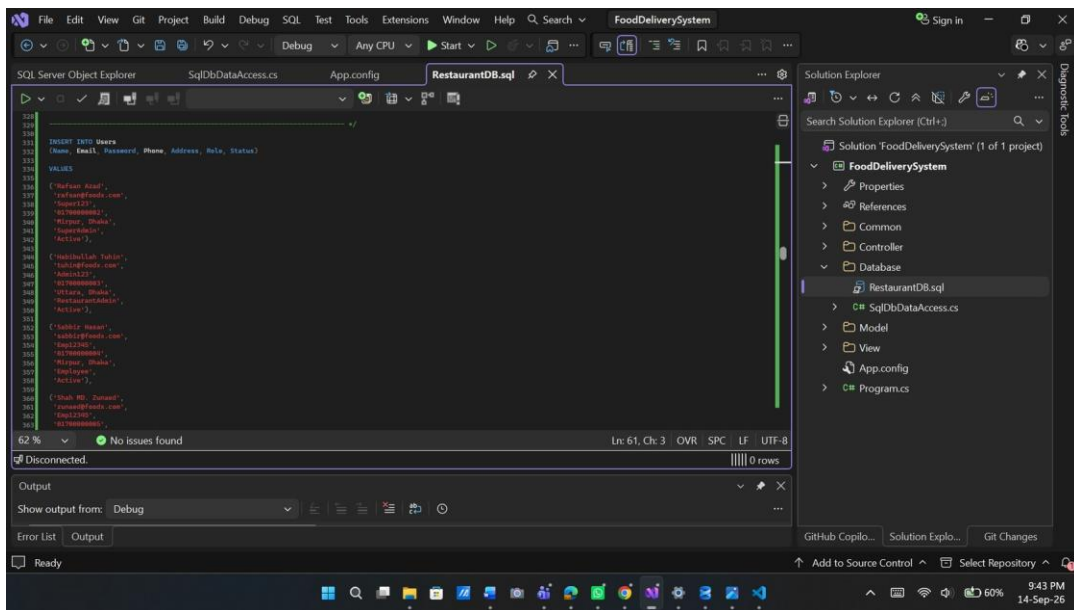
i) App.config – SQL Server connection string.



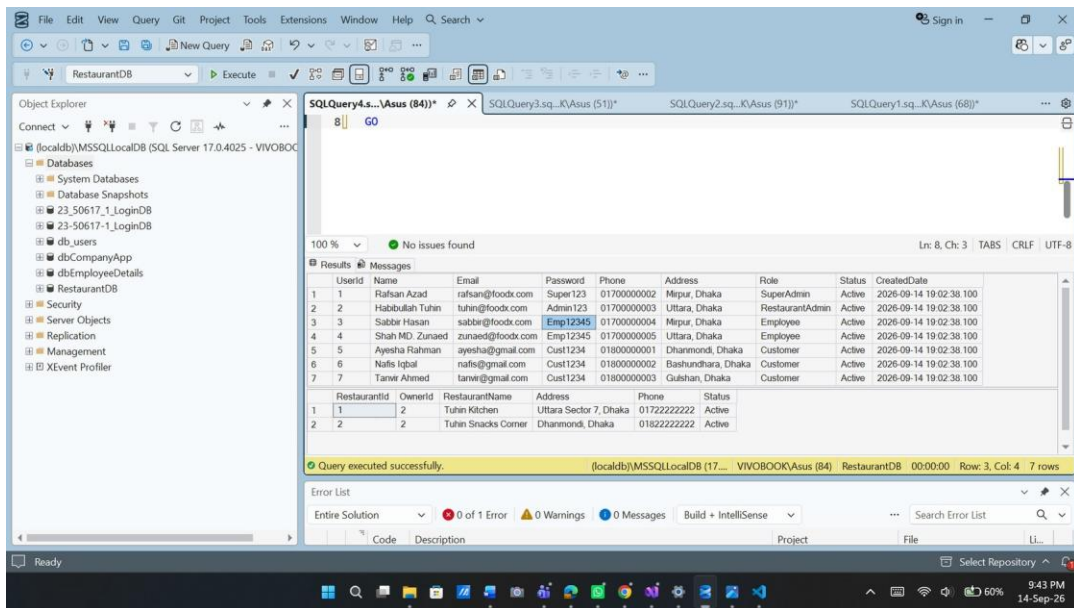
ii) RestaurantDB.sql – Users table creation script.



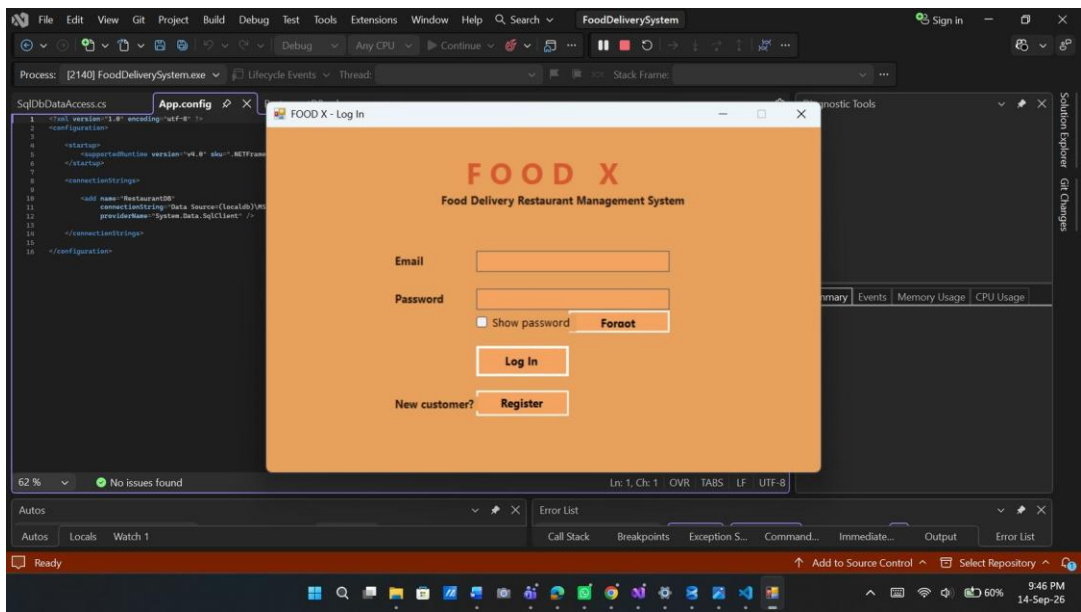
iii) Sample data insert for the Users table.



iv) Users and Restaurants data as stored in SQL Server.



v) System final run – FoodX login screen at start-up.



9. Features and Facilities

- **Role-Based Access:** Four distinct dashboards – Super Admin, Restaurant Admin, Employee, and Customer – each showing only the data and actions relevant to that role.
- **Multi-Restaurant Support:** A single Restaurant Admin account can own and manage more than one restaurant, each with its own menu, staff, and revenue figures.
- **Menu & Inventory Management:** Restaurant Admins add, edit and categorize food items, track stock levels, and receive low-stock warnings.
- **Shopping Cart & Checkout:** Customers search and filter the combined menu of all active restaurants, build a cart, and check out with a choice of payment method (Cash on Delivery, Card, or Mobile Banking).
- **Order Lifecycle Tracking:** Every order moves through Pending → Accepted → Preparing → Completed (or Cancelled), visible to both the customer and the assigned restaurant employee.
- **Invoicing & Reporting:** Customers can view a printable invoice for each order, while Restaurant Admins and the Super Admin can view sales, best-selling items, and platform-wide revenue reports.

10. Conclusion

FoodX brings together restaurant owners, kitchen staff, customers, and a platform-level Super Admin inside one coherent, database-driven desktop application. By modelling the real-world relationships between users, restaurants, categories, foods, carts, orders and payments as a properly normalized relational schema, and by exposing that schema through role-specific C# Windows Forms dashboards, the system demonstrates end-to-end order handling – from menu browsing to checkout, kitchen fulfilment, and invoicing.

Beyond satisfying the requirements of the Object-Oriented Programming 2 project, FoodX highlights good practice in GUI-based application design: separating concerns across Model, View, Controller and Database layers, enforcing data integrity with primary/foreign keys and CHECK constraints, and validating and verifying data at every step of the order pipeline. Overall, the project reflects a realistic, extensible foundation for a food-delivery platform.